CIS 434 · Social Media Analytics — Final Cheat Sheet

Text · Web · SNA | 70 MCQ · 90 min

1 · PIPELINE + BOW

tokenize → normalize (stem/lemma/stop-word/min_df) → vectorize (BoW or TF-IDF) → model

Concept	Definition
Tokenize	Split text into tokens. Always step #1.
BoW	Vector of word counts. Loses order.
Term-Doc Matrix	Rows=docs, cols=terms, cells=freq/binary.
Stem	Suffix chop, no dict: 'running'→'run'.
Lemmatize	Dict base form: 'feet'→'foot', 'better'→'good'.
Stop-word	Drop 'is','the','a' (low info).
min_df	Drop rare tokens (don't generalize).

```
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer
X = CountVectorizer(stop_words='english', min_df=2).fit_transform(corpus)
```

2 · TF-IDF · N-GRAM · POS

TF·IDF = (k/n) · log(N/df). k=term count in doc; n=#terms in doc; N=corpus; df=docs containing term. High when frequent here, rare in corpus.

☞ *1000 docs, doc has 25 terms, 'unstructured' appears 3× and in ■ of docs (df=200) → **(3·log 5)/25**.*

n-grams capture local order: 'not good'≠'good'. W tokens → **W–1 bi-grams**, **W–2 tri-grams**. Ex: 'Social media generates lots of unstructured data' (7) → **6 bi-grams**.

POS labels noun/verb/etc; preserves sentence-level partial order. Higher n + POS ⇒ very high-dim sparse matrix.

```
from sklearn.feature_extraction.text import TfidfVectorizer
vec = TfidfVectorizer(ngram_range=(1,2), min_df=2)
X = vec.fit_transform(corpus)
from nltk import pos_tag, word_tokenize
pos_tag(word_tokenize(text)) # [(word, tag), ...]
```

3 · SIMILARITY · CLUSTERING · LDA

Method	Note
Cosine sim	angle between vectors; ignores length. Default for text.
Jaccard	|A∩B|/|A∪B| (sets/binary).
Overlap	|A∩B|/min(|A|,|B|).
K-Means	Hard 1-cluster/doc, k centroids. Topic = top words of centroid.
Agglomerative	Bottom-up hierarchical merges.
LDA	Probabilistic: doc = mixture of topics; topic = distribution of words.
Cluster vs LDA	Cluster = 1 cluster/doc; LDA = overlapping topic membership.

K-Means steps: (1) pick k (2) init k centroids (3) assign each doc to nearest centroid (4) recompute centroids = mean of members (5) repeat until stable. **Topic words** = top-weighted terms in each centroid vector.

```
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.decomposition import LatentDirichletAllocation as LDA
km = KMeans(n_clusters=5).fit(X); lda = LDA(n_components=5).fit(X)
from sklearn.metrics.pairwise import cosine_similarity
cosine_similarity(X[0], X[1])
```

4 · SENTIMENT · LOGIT · VADER

Sentiment = classify tone (pos/neg/neu). Score = **weighted sum** of word presence × sentiment weight: **s = x·β**; label by cutoff (s > 0 → pos).

VADER = lexicon-based polarity (no training). Adjusts for caps, '!!!', booster/negation, slang. sid.polarity_scores(text) → {neg, neu, pos, **compound** ∈ [–1,1]}. ≥0.05 pos, ≤–0.05 neg.

Logit: P = 1/(1+e$^{-Xβ}$). β **= per-word sentiment weights** (interpretable). β learned by maximizing log-likelihood on labeled data; iterate until accuracy stops improving.

Train/test split mandatory ⇒ tests **generalization**. Without it, accuracy is inflated by overfit.

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
sid = SentimentIntensityAnalyzer(); sid.polarity_scores(text)
from sklearn.linear_model import LogisticRegression
model.fit(train_x, train_c); y_pred = model.predict(test_x)
acc = accuracy_score(test_c, y_pred) # (true, predicted)
```

5 · OTHER CLASSIFIERS

Model	Idea / formula
Naive Bayes	Bayes rule: P(c|x) ∝ P(x|c)·P(c). Assumes feature independence; ~70% acc baseline.
SVM	Find hyperplane w·x – b = 0 w/ max margin; support vectors = points on margin.
Decision Tree	Flow-chart splits until leaf has ~homogeneous labels. Interpretable; overfits.
Random Forest	Many trees + bagging (bootstrap aggregating) on rows+features → majority vote → cuts overfit.
Logit	Linear, interpretable β/term; equivalent to ANN w/ 0 hidden layers.

☞ *RF overfits (train■test) ⇒ **raise bootstrap subsampling** of features+rows. Don't deepen / don't drop trees / don't switch features.*

Fair contest: same train/test + same features; compare on **test accuracy**. **Overfit** = train acc > test acc (learned noise).

Improve model: refine rep (n-grams, min_df) · try different classifier · increase sample size.

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
RF = RandomForestClassifier(n_estimators=50, max_depth=3, bootstrap=True)
```

6 · EVAL METRICS & CONFUSION MATRIX

Metric	Formula / use
Confusion mat	TP / FP / FN / TN. Rows=true, cols=pred.
Accuracy	(TP+TN)/(all). Default; bad on imbalance.
Precision	TP/(TP+FP). Spam-style: avoid false alarm.
Recall (TPR)	TP/(TP+FN). Disease-screen: don't miss.
F1	2·P·R/(P+R). Harmonic mean.
FPR	FP/(FP+TN).
ROC	TPR (y) vs FPR (x) over thresholds; trade-off.
AUC	Area under ROC; higher = better separator.

```
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, roc_auc_score)
accuracy_score(y_true, y_pred) # ALWAYS (true, pred)
```

7 · NEURAL NETWORKS + IMAGES

Neuron: a = f(Σ w■x■ + b). **Activations:** ReLU max(0,x), sigmoid 1/(1+e^{-x}), tanh, softmax (multi-class). Add non-linearity.

vs Logit/SVM: stacks many **hidden layers** → learns non-linear hierarchical features (low → mid → high). Logit = ANN with 0 hidden layers.

Param count dominated by **1st layer** (depends on input size). 400×400×3 input → 1 hidden of 480k ≈ **230B params**; 1 hidden of 1k ≈ **480M**. **Deep+narrow ■ shallow+wide** for same capacity.

☞ ***MLPClassifier(hidden_layer_sizes=(2,3))** = **2 hidden layers**: 1st has 2 neurons, 2nd has 3. **(3,2)** = 3-then-2.*

Image: flatten 2D RGB to vector. 100×200×3 → **60000 dims = (60000,1)**. Then any classic ML applies.

DL needs big data: beats traditional only at large data; otherwise overfits. Pro: more functions. Con: more params, more data.

```
from sklearn.neural_network import MLPClassifier
DL = MLPClassifier(solver='lbfgs', hidden_layer_sizes=(3,2),
activation='relu', random_state=1)
DL.fit(train_x, train_c); accuracy_score(test_c, DL.predict(test_x))
```

8 · WORD EMBEDDING

Encoding	Properties
Index	word→int. Preserves order; no semantics.
One-hot	Binary vector/word; orthogonal ⇒ no similarity; sparse.
BoW	Sum of one-hots; loses order.
N-gram rep	Phrase-level local order.
POS rep	Order within short sentence.
W2V / GloVe	Dense, low-dim; distance ≈ semantic similarity.

☞ *Padded one-hot for K docs, max len L, vocab V → shape **(K, L, V)**. e.g. 123 docs, max 50, V=4000 → **(123, 50, 4000)**.*

Semantic arithmetic: king – man + woman ≈ queen. **Pre-trained** (Google W2V, Stanford GloVe) = dict {term: vec}; great for small/noisy data.

Embedding layer in NN = trainable dense vectors learned during training; standard front of LSTM/RNN.

9 · RNN + LSTM + SEQ2SEQ

Order matters for translation, chatbot, sentiment ⇒ use order-preserving rep (index/one-hot).

RNN: 1 unit reused across time-steps; hidden state carries memory. **A step is NOT a unit.** Params depend only on per-step input ⇒ huge savings vs flattened.

Vanishing/exploding gradient: backprop thru long seq fails ⇒ only later tokens learned. **LSTM** adds **memory cell + input/forget/output gates** ⇒ keeps long-range deps; learns earlier tokens. Built-in attention.

☞ ***Batch size** = #samples before weight update (memory). **Epoch** = 1 full pass. Need many epochs with batches so early batches re-seen after late batches.*

Padding: all docs same length L via pad_sequences(x, maxlen=L) (truncate longer, pad shorter w/ 0).

Seq2Seq: Encoder reads input seq → context vec → **Decoder** generates output seq token-by-token. START / END tokens. Translation, chatbot.

```
from keras.preprocessing.sequence import pad_sequences
x_train = pad_sequences(x_train, maxlen=80)
model = Sequential()
model.add(Embedding(V+1, 40, input_length=80))
model.add(LSTM(32, dropout=0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.fit(x_train, y, batch_size=32, epochs=3)
```

10 · ACTIVATIONS · LOSSES · OUTPUTS

Activation	Use
ReLU max(0,x)	Hidden layers default; cheap, no vanishing on positive side.
Sigmoid	Binary output; squashes to (0,1).
Tanh	Hidden, zero-centered; (–1,1).
Softmax	Multi-class output; probabilities sum 1.
Output / Loss	**Pair**
Binary	sigmoid + binary_crossentropy
Multinomial	softmax + categorical_crossentropy
Continuous	linear + MSE
Optimizer	adam (default), lbfgs (sklearn small)

11 · SHAPES & PARAM-COUNT CHEATS

Object	Shape / count
BoW matrix	(N_docs, V)
TF-IDF matrix	(N_docs, V)
Padded one-hot	(N_docs, L, V)
Embedded seq	(N_docs, L, d) — d■V
Image flattened	(H·W·C, 1) e.g. 100×200×3 = (60000,1)
MLP layer params	≈ in_dim × out_dim (+ biases)
1st-layer params	= V × neurons (dominates total)
RNN params	= per-step input × hidden (reused across t)

KEY QUIZ TRAPS — TEXT/DL

- Term-doc matrix represents **word frequencies across docs**.
- Lemmatization = **dictionary mapping** (not stemming).
- Cosine = **angle** between vectors.
- LDA improvement: **overlapping topic membership**.
- accuracy_score args = **(test_c, y_pred)**.
- Logit β = **sentiment weights per term**.
- Embedding > one-hot ⇒ **semantic similarity preserved**.
- LSTM solves **vanishing gradient**.
- RF overfit fix: **bootstrap aggregating**.
- hidden_layer_sizes=(2,3) → **2 layers, 2 then 3** neurons.
- A step in RNN is **not** a unit; one unit handles full seq.
- Test split's purpose: **evaluate generalization**.

12 · WEB SCRAPE (SCRAPY + XPATH)

HTML = nested tags w/ **id** (unique) + **class** (reusable). XPath queries the tree. CLI: scrapy crawl spider_name -o out.csv.

XPath / call	Meaning
//div[@class='c']/a	Direct <a> children of any div.c
//div[@class='c']//a	All <a> descendants of div.c (any depth)
/div[@class='c']/a	From root only (rarely useful)
.extract()	List of all matches
.extract_first()	First match or None
response.urljoin(rel)	Convert relative href → absolute

Wrapper pattern (multi-feature item): grab parent block, loop, pull each child by relative XPath.

```
def parse(self, response):
for w in response.xpath('//div[@class="post quote"]'):
q = w.xpath('span/text()').extract_first()
a = w.xpath('div[@class="source"]/text()').extract_first()
yield {'quote': q, 'author': a}
```

Pagination (same-type pages): next href → urljoin → **yield Request(url, callback=self.parse)**.
Hierarchical (different page types): multiple parse fns. Pass partial via Request(..., meta={...}); receive in lower fn via response.meta.

```
next_url = response.urljoin(
response.xpath('//div[@id="pagination"]/a/@href').extract()[-1])
yield Request(next_url, callback=self.parse)
yield Request(detail_url, callback=self.parse_lower,
meta={'Quote': q, 'Author': a})
```

☞ *Quiz: only **yield Request(url=..., callback=self.fn)** is correct — capital R, **self.**, no quotes.*

13 · NETWORK BASICS + REPRESENTATION

Network=nodes (vertices) + edges (ties). **Undirected** (FB friend, kinship) vs **Directed** (Twitter follow): tail→head.
Walk=any edge sequence. **Path**=no repeat nodes. **Cycle**=closed path. **Geodesic**=shortest path; geodesic distance = its length. **Component**=maximal connected subgraph. **Strongly connected**=path between every pair (directed). **Neighborhood**=directly connected nodes.

Representation	Form
Adjacency list	<code>{node: [neighbors]}</code>
Adjacency matrix	N×N binary; symmetric ⇔ undirected
Edge list	<code>[(u,v), ...]</code>

☞ *Max edges: undirected **n(n−1)/2**, directed **n(n−1)**. Directed graph has ½ **density** of its undirected version.*

```
import networkx as nx
G = nx.Graph(adj_list) # from dict
G = nx.from_numpy_array(A) # from matrix
G = nx.Graph(edges) # from edge list
G = nx.read_edgelist('g.edgelist', nodetype=int)
```

14 · NETWORK DIAGNOSIS (NETWORK-LEVEL)

Metric	Formula / meaning
Density	2|E|/(|V|(|V|−1)) undir; halved for directed.
Avg degree	Σ deg(v)/|V|.
Clustering (transitivity)	closed triplets / triplets (triangles).
Diameter	max shortest-path; undirected only; K-level→2K.
Connectivity	min nodes/edges to disconnect → robustness.
Reciprocity	sym edges/total (directed only).
Centralization	spread of centrality across nodes.

☞ *Same density ≠ same speed: **clustered edges = local groups** (slow spread); dispersed edges spread fast. Same clustering also ≠ same speed.*

```
nx.density(G) nx.transitivity(G)
nx.diameter(G) nx.node_connectivity(G)
nx.edge_connectivity(G) nx.reciprocity(G)
# Degree centralization:
max_d = max(dict(G.degree()).values())
centzn = sum(max_d - d for d in dict(G.degree()).values()) \
/ ((N-1)*(N-2))
```

15 · WORKED MINI-EXAMPLES

Density (5 edges, 4 nodes, undirected): max=4·3/2=6 → 5/6=**0.83**.
Node clustering: count closed triplets / total triplets in a node's *neighborhood*. Node connected to 2,3 with 2-3 edge → 1/1 = **1**. Triangle of 4 with center → coef **0** (no neighbor-neighbor edges).
Closeness of node v: 1/avg geodesic. Path A-B-C-D-E from B: 1/((1+1+2+3)/4)=4/7.
Reciprocity (5 directed edges, 2 reciprocal pairs=4 sym edges): 2/5 = 0.4.
Bi-grams of N tokens: N−1. **Tri-grams:** N−2.

16 · NODE CENTRALITY (INFLUENCERS)

Centrality	Captures / pick when…
Degree	# direct connections; reach immediate neighbors. In-deg=followers; out-deg=followi
Closeness	1/mean geodesic to all; speed of reach.
Betweenness	# geodesics through node; broker/gatekeeper; cuts flow.
Eigenvector	Connected to other influential nodes (recursive).
Clustering (node)	closed/total triplets in nbhd; local cohesion.

Structural hole = gap between sub-groups bridged by 1 node ⇒ **high betweenness**; broker controls info flow → competitive advantage.
☞ ***Seed picking by goal**: non-contagious + reach many → **degree**. Reach quickly → **closeness**. Reach other influencers → **eigenvector**. Counter campaign / cut flow → **betweenness**. Reach well-knit subgroups → seed inside them.*

```
nx.degree_centrality(G) nx.closeness_centrality(G)
nx.betweenness_centrality(G) nx.eigenvector_centrality(G)
nx.clustering(G)
top5 = sorted(c.items(), key=lambda x:x[1], reverse=True)[:5]
```

17 · COMMUNITY DETECTION

Method	How / Limit
Components	<code>nx.connected_components</code>; for disconnected communities.
Girvan-Newman	Iter remove edge w/ highest betweenness. Heavy compute. Keeps all nodes
Stoer-Wagner	Global min cut (max-flow); only 2 communities; also gives edge connectivity.
k-Core	Subgraph w/ every node deg ≥ k. Drops sparse nodes; can return 0/1 community. B
Louvain	Hierarchical greedy modularity max; scalable for large nets.

Louvain process: (1) each node = own community (2) **P1:** move node to neighbor community to ↑modularity (3) **P2:** collapse communities into super-nodes (4) repeat until no improvement.
Modularity Q: density intra-community vs inter. **Q**≈**0.7**=strong, **Q**≈**0.2**=weak. **Pre-partition** indicators: **cohesion+centralization** ⇒ does net *look* like it has communities?

```
from networkx.algorithms.community import girvan_newman
next(girvan_newman(G)) # first split
cut, parts = nx.stoer_wagner(G) # min cut + 2 parts
core = nx.k_core(G, k=2) # ≥2 connections
import community.community_louvain as cl
parts = cl.best_partition(G) # {node: community}
```

KEY QUIZ TRAPS — WEB/SNA

- //div[@class='c']/a = direct <a> children only.
- Correct: **yield Request(url='...', callback=self.parse_data)**.
- nx.transitivity / nx.clustering DON'T identify influential nodes (cohesion).
- **k-Core** is the only community method that *loses nodes*.
- **Stoer-Wagner** also yields edge connectivity.
- **Louvain** is best for large networks (greedy modularity).
- Modularity Q is *post-partition*; cohesion+centralization are *pre-partition*.
- Diameter undirected only; reciprocity directed only.
- 'Most influential' candidates = degree, betweenness, closeness, eigenvector — NOT clustering / transitivity.

FAST-RECALL CARDS

BoW vs One-hot: BoW = sum of one-hots, no order. One-hot keeps order, very high-dim.
TF-IDF higher when: common in *this* doc, rare in corpus.
Cosine vs Jaccard: cosine=angle (continuous); Jaccard=set overlap (binary).
K-Means vs LDA: hard vs soft; centroid vs distribution.
Logit β: word's contribution to sentiment (interpretable).
NB / SVM / DT / RF / Logit: compare on same train/test by accuracy.
MLP shape (a,b,c): 3 hidden layers a→b→c neurons.
Vanishing grad: RNN issue; LSTM gates fix it.
Batch×Epoch: small batch ⇒ more updates/epoch; many epochs needed.
Density vs **clustering**: density=overall edge frac; clustering=triangle frac.
Diameter: longest *shortest* path.
Reciprocity: directed only.
Centrality summary: degree (popular) · closeness (fast) · betweenness (broker) · eigenvector (well-connected to powerful).
Components = disconnected; **Girvan-Newman** = remove top-betweenness edges; **Stoer-Wagner** = global min cut; **k-Core** = strip sparse; **Louvain** = modularity, scalable.
Pre-partition: cohesion + centralization. **Post**: modularity Q.
Influencer pricing: nano 1k–10k · micro 10k–100k · mid 100k–500k · macro 500k–1M · mega 1M+.

MGP SCENARIO — WHICH NET / SEED?

Highly contagious + unlimited time + many seeds → big, well-connected net (high avg degree).
Very limited time → low-diameter net + high-closeness seed.
Not contagious + 1 seed, reach many → high-degree node in dense net.
Not contagious + counter rival post → high-betweenness node (cuts spread paths).
Reach other influencers first → high-eigenvector seed.
Reach well-connected sub-groups → seed inside dense communities (k-Core / Louvain).
Reach all in shortest time, contagious → small-diameter, high-clustering net.